

EduGenie — Complete Source Code

Generated from the supplied EduGenie project documentation. This PDF contains the complete source files created for the runnable project.

File: .env.example

```
# Required: create a Gemini API key in Google AI Studio.
GEMINI_API_KEY=your_gemini_api_key_here

# Current Gemini model. You can change this to another supported text model.
GEMINI_MODEL=gemini-3.8-flash

# The project documentation specifies a local LaMini-Flan-T5 explanation model.
# Keep false for the simplest setup. Set true only after installing requirements-local.txt.
USE_LOCAL_EXPLAINER=false
LOCAL_EXPLAINER_MODEL=MBZUAI/LaMini-Flan-T5-783M

MAX_INPUT_CHARS=30000
```

File: .gitignore

```
.venv/  
venv/  
__pycache__/  
*.py[cod]  
.pytest_cache/  
.env  
.DS_Store  
.vscode/
```

File: PROJECT_NOTES.md

Implementation notes

This project was generated from the supplied EduGenie documentation.

Documented requirements implemented:

- FastAPI backend
- HTML/CSS frontend
- Q&A
- Concept explanation
- Quiz generation
- Summary generation
- Learning recommendations
- `/qa`
- `/explain`
- `/quiz`
- `/summarize`
- `/learn/recommendations`
- Uvicorn local deployment
- Jinja2 templates
- Gemini integration
- Optional LaMini-Flan-T5 local explanation mode

The supplied documentation does not include complete source code, so this repository implements the missing code rather than reproducing una

Security basics:

- API keys are loaded from `.env`.
- `.env` is ignored by Git.
- User input is validated with Pydantic.
- The frontend escapes generated text before inserting it into HTML.
- Quiz JSON is validated before being returned by the API.

Production considerations:

- Add authentication/rate limiting before public deployment.
- Add logging/monitoring.
- Consider request quotas and cost controls.
- Add a database if progress tracking is introduced.
- Add stronger content/safety policies for a public educational product.

File: README.md

EduGenie – Google Gemini Powered Learning Assistant

EduGenie is a lightweight educational assistant based on the project documentation supplied with this project.

It provides:

- Question answering
- Simplified concept explanations
- 3-question multiple-choice quiz generation
- Educational text summarization
- Personalized learning recommendations

Architecture

```
```text
EduGenie/
 main.py
 config.py
 ai_client.py
 qna.py
 explanation_module.py
 quiz_module.py
 summary_module.py
 learning_path.py
 requirements.txt
 requirements-local.txt
 .env.example
 .gitignore
 templates/
 index.html
 static/
 style.css
 tests/
 test_api.py
```
```

The supplied documentation describes FastAPI + HTML/CSS, five REST endpoints, Gemini for Q&A/quiz/summary/learning paths, and LaMini-Flan-T5

Important model update

The supplied document names `gemini-1.5-pro`. That model has since been shut down by Google. This project therefore uses the current Google

The default in this project is `gemini-3.8-flash`. If your Google AI account exposes a different current text model, change `GEMINI\_MODEL` in

1. Open the project in VS Code

Open this folder:

```
```text
EduGenie
```
```

2. Create a virtual environment

Windows PowerShell:

```
```powershell
python -m venv .venv
.venv\Scripts\Activate.ps1
```
```

Windows Command Prompt:

```
```bat
python -m venv .venv
.venv\Scripts\activate
```
```

macOS/Linux:

```
```bash
python3 -m venv .venv
source .venv/bin/activate
```
```

3. Install dependencies

```
```bash
python -m pip install --upgrade pip
pip install -r requirements.txt
```
```

4. Configure Gemini

Copy:

```
```text
.env.example
```
```

to:

```
```text
.env
```
```

Then replace:

```
```text
GEMINI_API_KEY=your_gemini_api_key_here
```
```

with your actual Gemini API key.

Do not commit `.env` to Git.

5. Start EduGenie

```
```bash
uvicorn main:app --reload
```
```

Open:

```
```text
http://127.0.0.1:8000
```
```

FastAPI API documentation is available at:

```
```text
http://127.0.0.1:8000/docs
```
```

6. Test

Run:

```
```bash
pytest -q
```
```

The included tests verify that the application starts, the home page renders, and the health endpoint works.

For full AI tests, enter a valid Gemini API key and use the browser interface.

API endpoints

POST /qa

```
```json
{
 "question": "Which is the largest ocean?"
}
```
```

POST /explain

```
```json
{
 "topic": "Pythagoras theorem"
}
```
```

POST /quiz

```
```json
{
 "text": "Photosynthesis is the process by which green plants..."
}
```
```

The response contains exactly three questions with four options each.

POST /summarize

```
```json
{
 "text": "Long educational passage..."
}
```
```

POST /learn/recommendations

```
```json
{
 "topic": "SQL",
 "level": "beginner"
}
```
```

Optional local LaMini explanation model

The documentation proposes `MBZUAI/LaMini-Flan-T5-783M` for concept explanation.

To enable it:

```
```bash
pip install -r requirements-local.txt
```
```

Then set:

```
```text
USE_LOCAL_EXPLAINER=true
```
```

The first local-model use may download model files and can require substantial disk space and RAM.

When `USE\_LOCAL\_EXPLAINER=false`, explanations use Gemini instead, which makes the default setup much simpler.

Troubleshooting

"GEMINI_API_KEY is missing"

Make sure `.env` exists beside `main.py` and contains:

```
```text
GEMINI_API_KEY=...
```
```

Restart Uvicorn after changing `.env`.

Model not available

Change:

```
```text
GEMINI_MODEL=gemini-3.8-flash
```
```

to a current text-generation model available to your Gemini API account.

Port already in use

Use another port:

```
```bash
uvicorn main:app --reload --port 8001
```
```

Then open:

```
```text
http://127.0.0.1:8001
```
```

File: ai_client.py

```
from functools import lru_cache

from google import genai

from config import GEMINI_API_KEY, GEMINI_MODEL

class AIConfigurationError(RuntimeError):
    """Raised when the Gemini client cannot be configured."""

@lru_cache(maxsize=1)
def get_client() -> genai.Client:
    if not GEMINI_API_KEY:
        raise AIConfigurationError(
            "GEMINI_API_KEY is missing. Add it to the .env file."
        )
    return genai.Client(api_key=GEMINI_API_KEY)

def generate_text(prompt: str, *, temperature: float = 0.3) -> str:
    client = get_client()
    response = client.models.generate_content(
        model=GEMINI_MODEL,
        contents=prompt,
        config={
            "temperature": temperature,
        },
    )
    text = getattr(response, "text", None)
    if not text:
        raise RuntimeError("Gemini returned an empty response.")
    return text.strip()
```

File: config.py

```
import os
from pathlib import Path

from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent
load_dotenv(BASE_DIR / ".env")

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY", "").strip()
GEMINI_MODEL = os.getenv("GEMINI_MODEL", "gemini-3.8-flash").strip()

# The original document proposes LaMini-Flan-T5-783M for explanations.
# Keep this optional because the local model can require significant RAM/disk.
USE_LOCAL_EXPLAINER = os.getenv("USE_LOCAL_EXPLAINER", "false").lower() == "true"
LOCAL_EXPLAINER_MODEL = os.getenv(
    "LOCAL_EXPLAINER_MODEL",
    "MBZUAI/LaMini-Flan-T5-783M",
).strip()

MAX_INPUT_CHARS = int(os.getenv("MAX_INPUT_CHARS", "30000"))
```

File: explanation_module.py

```
from ai_client import generate_text
from config import LOCAL_EXPLAINER_MODEL, USE_LOCAL_EXPLAINER

_local_pipeline = None

def _get_local_pipeline():
    global _local_pipeline
    if _local_pipeline is None:
        try:
            from transformers import pipeline
        except ImportError as exc:
            raise RuntimeError(
                "Local explanation mode requires transformers and torch. "
                "Install the optional dependencies from requirements-local.txt."
            ) from exc

        _local_pipeline = pipeline(
            "text2text-generation",
            model=LOCAL_EXPLAINER_MODEL,
        )
    return _local_pipeline

def _local_explanation(topic: str) -> str:
    generator = _get_local_pipeline()
    prompt = (
        "Explain this educational topic simply for a beginner. "
        "Use a short definition, an intuitive explanation, and one example. "
        f"Topic: {topic}"
    )
    result = generator(prompt, max_new_tokens=220, do_sample=False)
    return result[0]["generated_text"].strip()

def explain_concept(topic: str) -> str:
    if USE_LOCAL_EXPLAINER:
        return _local_explanation(topic)

    prompt = f"""
You are EduGenie's concept explanation tutor.

Explain the following topic to a beginner:
{topic}

Structure the answer as:
1. Simple definition
2. How it works
3. Easy example
4. One key point to remember

Keep the language clear and concise.
"""
    return generate_text(prompt, temperature=0.25)
```

File: learning_path.py

```
from ai_client import generate_text

def get_learning_recommendations(topic: str, level: str = "beginner") -> str:
    prompt = f"""
    Create a structured learning path for the topic "{topic}".

    Learner level: {level}

    Include:
    1. Learning goal
    2. Beginner foundations
    3. Intermediate concepts
    4. Advanced concepts
    5. A suggested sequence and approximate timeline
    6. Practice activities
    7. Useful resource types (videos, documentation, books, articles)
    8. A simple way to check progress

    Keep it practical and learner-friendly. Do not invent specific URLs.
    """
    return generate_text(prompt, temperature=0.35)
```

File: main.py

```
from pathlib import Path
from typing import Literal

from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel, Field

from qna import answer_question
from explanation_module import explain_concept
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

BASE_DIR = Path(__file__).resolve().parent

app = FastAPI(
    title="EduGenie",
    description="Google Gemini powered learning assistant",
    version="1.0.0",
)

app.mount("/static", StaticFiles(directory=BASE_DIR / "static"), name="static")
templates = Jinja2Templates(directory=BASE_DIR / "templates")

class TextRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=30000)

class QARequest(BaseModel):
    question: str = Field(..., min_length=1, max_length=10000)

class ExplainRequest(BaseModel):
    topic: str = Field(..., min_length=1, max_length=10000)

class QuizRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=30000)

class LearningPathRequest(BaseModel):
    topic: str = Field(..., min_length=1, max_length=5000)
    level: Literal["beginner", "intermediate", "advanced"] = "beginner"

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="index.html",
        context={"title": "EduGenie"},
    )

@app.get("/health")
async def health():
    return {"status": "ok", "service": "EduGenie"}

@app.post("/qa")
async def qa(payload: QARequest):
    try:
        return {"answer": answer_question(payload.question)}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc

@app.post("/explain")
async def explain(payload: ExplainRequest):
    try:
        return {"explanation": explain_concept(payload.topic)}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc

@app.post("/quiz")
async def quiz(payload: QuizRequest):
    try:
        return {"quiz": generate_quiz(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc

@app.post("/summarize")
async def summarize(payload: TextRequest):
    try:
```

```
        return {"summary": summarize_text(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc

@app.post("/learn/recommendations")
async def learning_recommendations(payload: LearningPathRequest):
    try:
        return {
            "recommendations": get_learning_recommendations(
                payload.topic, payload.level
            )
        }
    except Exception as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc
```

File: qna.py

```
from ai_client import generate_text

SYSTEM_PROMPT = """
You are EduGenie, a student-friendly educational assistant.
Answer the student's question accurately and concisely.
Explain unfamiliar terms in simple language.
Use short headings or bullet points when useful.
Do not invent sources or facts. If the question is ambiguous, state the
assumption you are making.
"""

def answer_question(question: str) -> str:
    prompt = f"""
{SYSTEM_PROMPT}

Student question:
{question}

Give a clear educational answer suitable for a learner.
"""
    return generate_text(prompt, temperature=0.2)
```

File: quiz_module.py

```
import json
import re
from typing import List

from pydantic import BaseModel, Field, ValidationError

from ai_client import generate_text


class QuizQuestion(BaseModel):
    question: str
    options: List[str] = Field(min_length=4, max_length=4)
    correct_answer: int = Field(ge=0, le=3)
    explanation: str


class Quiz(BaseModel):
    questions: List[QuizQuestion] = Field(min_length=3, max_length=3)


def clean_json_block(text: str) -> str:
    text = text.strip()
    text = re.sub(r"^\s*(?:json)?\s*$", "", text, flags=re.IGNORECASE)
    text = re.sub(r"\s*\s*$", "", text)
    return text.strip()


def generate_quiz(passage: str) -> dict:
    prompt = f"""
Create exactly 3 multiple-choice questions from the educational passage below.

Requirements:
- Exactly 3 questions.
- Exactly 4 options per question.
- correct_answer is a zero-based option index: 0, 1, 2, or 3.
- Include a short explanation for each answer.
- Questions must be answerable from the passage.
- Return ONLY valid JSON in this exact shape:
{{
  "questions": [
    {{
      "question": "string",
      "options": ["A", "B", "C", "D"],
      "correct_answer": 0,
      "explanation": "string"
    }}
  ]
}}
"""

    Passage:
    {passage}
    """

    raw = generate_text(prompt, temperature=0.2)
    cleaned = clean_json_block(raw)

    try:
        parsed = json.loads(cleaned)
        quiz = Quiz.model_validate(parsed)
    except (json.JSONDecodeError, ValidationError) as exc:
        raise RuntimeError(
            "Gemini returned quiz data that could not be validated as the "
            "required 3-question JSON format."
        ) from exc

    return quiz.model_dump()
```

File: requirements-local.txt

```
-r requirements.txt  
torch>=2.2  
transformers>=4.45  
sentencepiece>=0.2
```

File: requirements.txt

```
fastapi>=0.115,<1.0
uvicorn[standard]>=0.34,<1.0
jinja2>=3.1,<4.0
python-dotenv>=1.0,<2.0
google-genai>=1.0,<2.0
pydantic>=2.8,<3.0
httpx>=0.27,<1.0
pytest>=8.0,<9.0
```

File: run_mac_linux.sh

```
#!/usr/bin/env bash
set -e
if [ ! -d ".venv" ]; then
    python3 -m venv .venv
fi
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
if [ ! -f ".env" ]; then cp .env.example .env; fi
uvicorn main:app --reload
```

File: run_windows.bat

```
@echo off
if not exist .env (
  python -m venv .env
)
call .env\Scripts\activate
python -m pip install --upgrade pip
pip install -r requirements.txt
if not exist .env copy .env.example .env
uvicorn main:app --reload
```

File: static/style.css

```
:root {
  font-family: Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  color: #18212f;
  background: #f5f7fb;
  line-height: 1.6;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  background:
    radial-gradient(circle at top left, #e7efff 0, transparent 35%),
    #f5f7fb;
}

.app-shell {
  width: min(920px, calc(100% - 32px));
  margin: 0 auto;
  padding: 42px 0 30px;
}

.hero {
  margin-bottom: 26px;
}

.brand {
  display: flex;
  gap: 14px;
  align-items: center;
}

.logo {
  width: 52px;
  height: 52px;
  border-radius: 16px;
  display: grid;
  place-items: center;
  font-size: 25px;
  font-weight: 800;
  background: #18212f;
  color: white;
  box-shadow: 0 12px 30px rgba(24, 33, 47, 0.15);
}

.eyebrow {
  margin: 0;
  font-size: 11px;
  font-weight: 800;
  letter-spacing: 0.15em;
  opacity: 0.58;
}

h1 {
  margin: 0;
  font-size: clamp(34px, 7vw, 54px);
  line-height: 1;
}

.hero-copy {
  max-width: 680px;
  color: #5b6574;
  margin: 18px 0 0;
  font-size: 17px;
}

.card,
.result-card {
  background: rgba(255, 255, 255, 0.94);
  border: 1px solid #e1e6ef;
  border-radius: 22px;
  padding: 24px;
  box-shadow: 0 18px 55px rgba(32, 48, 80, 0.08);
  margin-bottom: 18px;
}

label {
  display: block;
  font-size: 14px;
  font-weight: 750;
  margin: 0 0 8px;
}

select,
textarea {
  width: 100%;
```

```

border: 1px solid #d5dce7;
border-radius: 13px;
background: white;
padding: 13px 14px;
font: inherit;
color: inherit;
outline: none;
margin-bottom: 18px;
}

select:focus,
textarea:focus {
border-color: #7288ff;
box-shadow: 0 0 0 4px rgba(114, 136, 255, 0.12);
}

textarea {
resize: vertical;
min-height: 180px;
}

button {
border: 0;
border-radius: 13px;
padding: 13px 18px;
font: inherit;
font-weight: 750;
cursor: pointer;
}

form > button {
width: 100%;
background: #18212f;
color: white;
}

button:disabled {
cursor: wait;
opacity: 0.65;
}

.secondary {
background: #eef2f8;
color: #253246;
padding: 8px 12px;
}

.result-header {
display: flex;
justify-content: space-between;
align-items: center;
}

.result-header h2 {
margin: 0 0 12px;
font-size: 20px;
}

.result {
border-radius: 14px;
background: #f7f9fc;
padding: 18px;
min-height: 120px;
color: #2f3a4a;
}

.result.empty {
color: #8893a3;
display: grid;
place-items: center;
}

.plain-response {
white-space: normal;
}

.error {
color: #a32929;
background: #fff0f0;
padding: 12px;
border-radius: 10px;
}

.quiz-question {
border-bottom: 1px solid #e1e6ef;
padding: 4px 0 20px;
margin-bottom: 20px;
}

.quiz-question:last-child {
border-bottom: 0;
margin-bottom: 0;
}

```

```

}

.quiz-question h3 {
  font-size: 16px;
  margin: 0 0 12px;
}

.options {
  display: grid;
  gap: 9px;
}

.option {
  text-align: left;
  background: white;
  border: 1px solid #d9e0eb;
  color: #263246;
}

.option:hover:not(:disabled) {
  background: #f0f4ff;
}

.option.correct {
  border-color: #2f8f57;
  background: #eaf8ef;
}

.option.wrong {
  border-color: #bd4a4a;
  background: #fff0f0;
}

.quiz-feedback {
  margin-top: 10px;
  font-weight: 700;
  font-size: 14px;
}

.spinner {
  width: 16px;
  height: 16px;
  display: inline-block;
  margin-left: 7px;
  vertical-align: -3px;
  border: 2px solid rgba(255,255,255,.45);
  border-top-color: white;
  border-radius: 50%;
  animation: spin .8s linear infinite;
}

.hidden {
  display: none !important;
}

footer {
  text-align: center;
  color: #8a94a4;
  font-size: 13px;
  padding: 14px;
}

@keyframes spin {
  to { transform: rotate(360deg); }
}

@media (max-width: 600px) {
  .app-shell {
    width: min(100% - 20px, 920px);
    padding-top: 22px;
  }

  .card,
  .result-card {
    padding: 17px;
    border-radius: 17px;
  }
}

```

File: summary_module.py

```
from ai_client import generate_text

def summarize_text(text: str) -> str:
    prompt = f"""
Summarize the following educational text for quick revision.

Requirements:
- Preserve the important facts and relationships.
- Remove repetition and unnecessary detail.
- Use simple language.
- Prefer a short overview followed by bullet points.
- Do not add facts that are not present in the source.

Text:
{text}
"""
    return generate_text(prompt, temperature=0.2)
```

File: templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EduGenie – AI Learning Assistant</title>
  <link rel="stylesheet" href="{{ url_for('static', path='/style.css') }}">
</head>
<body>
  <main class="app-shell">
    <header class="hero">
      <div class="brand">
        <div class="logo">E</div>
        <div>
          <p class="eyebrow">AI LEARNING ASSISTANT</p>
          <h1>EduGenie</h1>
        </div>
      </div>
      <p class="hero-copy">
        Ask questions, simplify concepts, generate quizzes, summarize notes,
        and build a personalized learning path.
      </p>
    </header>

    <section class="card">
      <form id="edu-form">
        <label for="task">Choose a learning task</label>
        <select id="task" name="task">
          <option value="qa">Ask a Question</option>
          <option value="explain">Explain a Concept</option>
          <option value="quiz">Generate Quiz</option>
          <option value="summarize">Summarize Text</option>
          <option value="learn">Learning Recommendations</option>
        </select>

        <div id="level-wrap" class="hidden">
          <label for="level">Learning level</label>
          <select id="level">
            <option value="beginner">Beginner</option>
            <option value="intermediate">Intermediate</option>
            <option value="advanced">Advanced</option>
          </select>
        </div>

        <label for="input-text" id="input-label">Your question</label>
        <textarea
          id="input-text"
          rows="9"
          maxlength="30000"
          placeholder="Example: Which is the largest ocean?"
          required
        ></textarea>

        <button type="submit" id="submit-btn">
          <span id="button-text">Ask EduGenie</span>
          <span id="spinner" class="spinner hidden"></span>
        </button>
      </form>
    </section>

    <section class="result-card">
      <div class="result-header">
        <h2>Result</h2>
        <button id="copy-btn" class="secondary hidden" type="button">Copy</button>
      </div>
      <div id="result" class="result empty">
        Your EduGenie response will appear here.
      </div>
    </section>

    <footer>
      EduGenie · FastAPI + Google Gemini · Educational support tool
    </footer>
  </main>

  <script>
    const form = document.getElementById("edu-form");
    const task = document.getElementById("task");
    const input = document.getElementById("input-text");
    const label = document.getElementById("input-label");
    const levelWrap = document.getElementById("level-wrap");
    const level = document.getElementById("level");
    const result = document.getElementById("result");
    const submitBtn = document.getElementById("submit-btn");
    const buttonText = document.getElementById("button-text");
    const spinner = document.getElementById("spinner");
    const copyBtn = document.getElementById("copy-btn");

    const taskInfo = {
```

```

qa: {
  label: "Your question",
  placeholder: "Example: Which is the largest ocean?",
  button: "Ask EduGenie"
},
explain: {
  label: "Topic to explain",
  placeholder: "Example: Explain the Pythagoras theorem simply.",
  button: "Explain Concept"
},
quiz: {
  label: "Topic or passage",
  placeholder: "Paste a topic or educational passage to create 3 MCQs.",
  button: "Generate Quiz"
},
summarize: {
  label: "Text to summarize",
  placeholder: "Paste the educational passage you want summarized.",
  button: "Summarize"
},
learn: {
  label: "Topic to learn",
  placeholder: "Example: SQL",
  button: "Build Learning Path"
}
};

function updateForm() {
  const info = taskInfo[task.value];
  label.textContent = info.label;
  input.placeholder = info.placeholder;
  buttonText.textContent = info.button;
  levelWrap.classList.toggle("hidden", task.value !== "learn");
}

function escapeHtml(value) {
  return value
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
    .replaceAll("'", "&quot;")
    .replaceAll('"', "&quot;");
}

function renderText(text) {
  return `<div class="plain-response">${escapeHtml(text)}
    .replace(/\n/g, "<br>")</div>`;
}

function renderQuiz(quiz) {
  return `
<div class="quiz">
  ${quiz.questions.map((q, i) => `
    <article class="quiz-question" data-correct="${q.correct_answer}">
      <h3>${i + 1}. ${escapeHtml(q.question)}</h3>
      <div class="options">
        ${q.options.map((option, j) => `
          <button type="button" class="option" data-index="${j}">
            ${String.fromCharCode(65 + j)}. ${escapeHtml(option)}
          </button>
        `).join("")}
      </div>
      <div class="quiz-feedback hidden"></div>
    </article>
  `).join("")}
</div>
`;
}

function attachQuizHandlers() {
  document.querySelectorAll(".quiz-question").forEach(question => {
    const correct = Number(question.dataset.correct);
    const feedback = question.querySelector(".quiz-feedback");
    question.querySelectorAll(".option").forEach(option => {
      option.addEventListener("click", () => {
        const selected = Number(option.dataset.index);
        question.querySelectorAll(".option").forEach(btn => {
          btn.disabled = true;
          btn.classList.remove("correct", "wrong");
        });
        if (selected === correct) {
          option.classList.add("correct");
          feedback.textContent = "Correct! ■";
        } else {
          option.classList.add("wrong");
          question.querySelector(
            `option[data-index="${correct}"]`
          ).classList.add("correct");
          feedback.textContent = "Not quite. The highlighted option is correct.";
        }
        feedback.classList.remove("hidden");
      });
    });
  });
}

```

```

    });
  });
}

task.addEventListener("change", updateForm);

copyBtn.addEventListener("click", async () => {
  await navigator.clipboard.writeText(result.innerText);
  copyBtn.textContent = "Copied";
  setTimeout(() => copyBtn.textContent = "Copy", 1200);
});

form.addEventListener("submit", async (event) => {
  event.preventDefault();

  const value = input.value.trim();
  if (!value) return;

  submitBtn.disabled = true;
  spinner.classList.remove("hidden");
  result.classList.remove("empty");
  result.textContent = "EduGenie is thinking...";
  copyBtn.classList.add("hidden");

  let endpoint;
  let payload;

  if (task.value === "qa") {
    endpoint = "/qa";
    payload = { question: value };
  } else if (task.value === "explain") {
    endpoint = "/explain";
    payload = { topic: value };
  } else if (task.value === "quiz") {
    endpoint = "/quiz";
    payload = { text: value };
  } else if (task.value === "summarize") {
    endpoint = "/summarize";
    payload = { text: value };
  } else {
    endpoint = "/learn/recommendations";
    payload = { topic: value, level: level.value };
  }

  try {
    const response = await fetch(endpoint, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload)
    });

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.detail || "Request failed.");
    }

    if (task.value === "qa") {
      result.innerHTML = renderText(data.answer);
    } else if (task.value === "explain") {
      result.innerHTML = renderText(data.explanation);
    } else if (task.value === "quiz") {
      result.innerHTML = renderQuiz(data.quiz);
      attachQuizHandlers();
    } else if (task.value === "summarize") {
      result.innerHTML = renderText(data.summary);
    } else {
      result.innerHTML = renderText(data.recommendations);
    }

    copyBtn.classList.remove("hidden");
  } catch (error) {
    result.innerHTML = `<div class="error">${escapeHtml(error.message)}</div>`;
  } finally {
    submitBtn.disabled = false;
    spinner.classList.add("hidden");
  }
});

updateForm();
</script>
</body>
</html>

```

File: tests/test_api.py

```
from fastapi.testclient import TestClient
import main

client = TestClient(main.app)

def test_home():
    response = client.get("/")
    assert response.status_code == 200
    assert "EduGenie" in response.text

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json()["status"] == "ok"
```